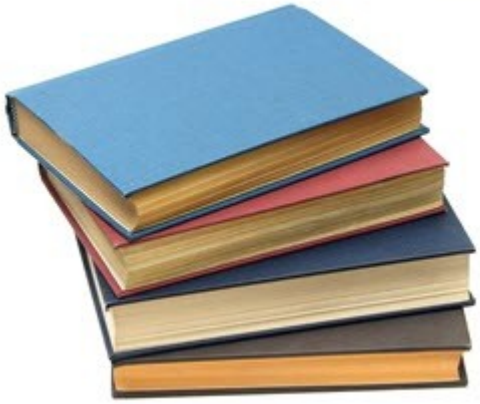
Introduction to the stack data structure

A stack is a data structure that holds a list of elements. A stack works based on the LIFO principle i.e., Last In, First out, meaning that the most recently added element is the first one to remove.

A stack has two main operations that occur only at the top of the stack: push and pop. The push operation places an element at the top of stack whereas the pop operation removes an element from the top of the stack.

The name *stack* comes from the analogy to a set of physical items e.g., DVD disc, books, stacked on top each other.



Operations on Stack

Push : Push is operation which inserts the data in to the stack.
Pop : Pop is a operation which removes the data from the stack.
Peek : Peek returns the data pointed by the top of the Stack.

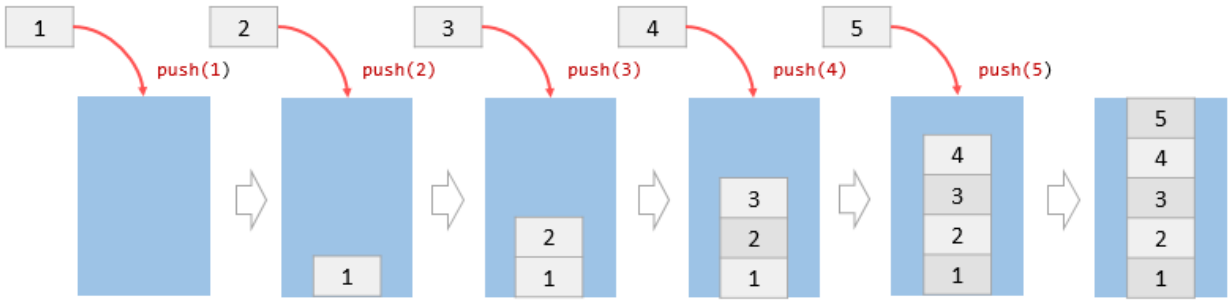
Algorithm for Push Operation

In a stack, push() is a function used to insert an element into the stack. In a stack, the new element is always inserted at **top** position. Push function takes one integer value as parameter and inserts that value into the stack. We can use the following steps to push an element on to the stack...

- Step 1 - Check whether **stack** is **FULL**. (**top == SIZE-1**)
- Step 2 - If it is **FULL**, then display **"Stack is FULL!!! Insertion is not possible!!!"** and terminate the function.

- Step 3 - If it is **NOT FULL**, then increment **top** value by one (**top++**) and set stack[top] to value (**stack[top] = value**).

The following figure illustrates each step in the above algorithm.

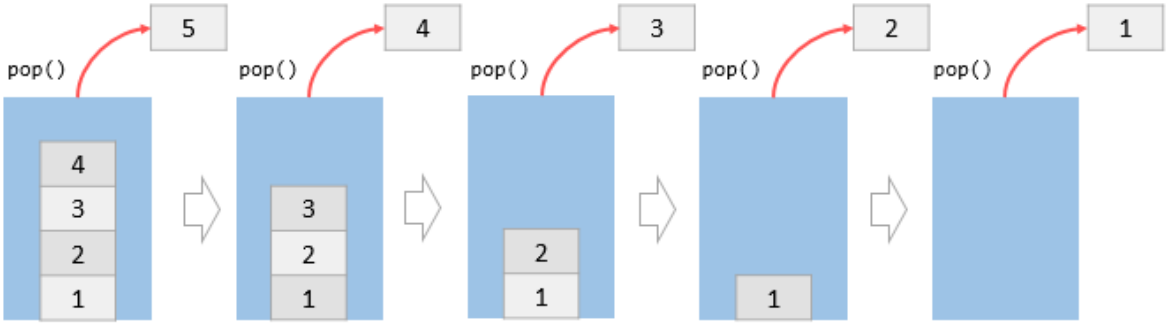


Algorithm for Pop operation

In a stack, pop() is a function used to delete an element from the stack. In a stack, the element is always deleted from **top** position. Pop function does not take any value as parameter. We can use the following steps to pop an element from the stack...

- Step 1 - Check whether **stack** is **EMPTY**. (**top == -1**)
- Step 2 - If it is **EMPTY**, then display **"Stack is EMPTY!!! Deletion is not possible!!!"** and terminate the function.
- Step 3 - If it is **NOT EMPTY**, then delete **stack[top]** and decrement **top** value by one (**top--**).

The following figure illustrates each step in the above algorithm.



Initially, the stack has 5 elements. The pop() method removes the element at the end of the array i.e., at the top of the stack one at a time. After five operations, the stack is empty

Analysis of Stack Operations

Below mentioned are the time complexities for various operations that can be performed on the Stack data structure.

Push Operation : O(1)

Pop Operation : O(1)

Top Operation : O(1)

Search Operation : O(n)

The time complexities for push() and pop() functions are O(1) because we always have to insert or remove the data from the top of the stack, which is a one step process.

Applications of Stack

- **Expression Evaluation**

  Stack is used to evaluate prefix, postfix and infix expressions.

- **Expression Conversion**

  An expression can be represented in prefix, postfix or infix notation. Stack can be used to convert one form of expression to another.

- **Syntax Parsing**

  Many compilers use a stack for parsing the syntax of expressions, program blocks etc. before translating into low level code.

Vector India

- **Backtracking**

  Suppose we are finding a path for solving maze problem. We choose a path and after following it we realize that it is wrong. Now we need to go back to the beginning of the path to start with new path. This can be done with the help of stack.

- **Parenthesis Checking**

  Stack is used to check the proper opening and closing of parenthesis.

- **String Reversal**

  Stack is used to reverse a string. We push the characters of string one by one into stack and then pop character from stack.

- **Function Call**

  Stack is used to keep information about the active functions or subroutines.

There are various other applications of stack. I have mentioned some of them.

Note :
Reference Books : Taken contents and diagrams from various websites.

Vector India